# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

## 1. End-to-End Workflow Description

The Academic Research Paper Assistant integrates PDF processing, text chunking, embedding generation, vector storage, document retrieval, and LLM-based answer generation into a single workflow.

The workflow begins when a user uploads an academic PDF through the Streamlit web interface. The PDF loader extracts text from each page and preserves the corresponding page numbers. The extracted text is then divided into smaller chunks using LangChain's RecursiveCharacterTextSplitter.

The application uses the local nomic-embed-text model through Ollama to generate embeddings for the document chunks. These embeddings, along with page numbers and source identifiers, are stored in a local ChromaDB vector store.

After processing the document, the user enters a natural-language question. The application converts the question into an embedding and performs a similarity of search to retrieve the most relevant document passages. The retrieved passages are combined with their page information and provided as context to the Llama 3.2 model through Ollama.

The LLM generates an answer based on the retrieved context. Finally, the Streamlit interface displays the generated answer along with the retrieved passages and their corresponding page numbers.

This integrated workflow demonstrates the initial two-step Retrieval-Augmented Generation (RAG) approach, where relevant document passages are retrieved first and then provided to the LLM for answer generation.

## 2. Components Integrated

The following components were integrated to implement the end-to-end RAG workflow.

| Component | File / Technology | Responsibility |
|---|---|---|
| User Interface | src/app.py — Streamlit | Provides PDF upload, question input, generated answers, and retrieved passage display. |
| PDF Processing | src/ingestion/pdf_loader.py — pypdf | Extracts text from PDF pages and preserves page numbers. |

| | | |
|---|---|---|
| Text Chunking | src/ingestion/text_splitter.py — LangChain | Divides extracted text into smaller chunks of 1,000 characters with 200-character overlap. |
| Embedding Generation | src/ingestion/embedding.py — Ollama | Use the local nomic-embed-text model to generate embeddings. |
| Vector Storage and Retrieval | src/retrieval/vector_store.py — ChromaDB | Stores document embeddings and metadata and retrieves relevant passages through similarity search. |
| Answer Generation | src/generation/answer_generator.py — LangChain and Ollama | Sends retrieved context to the local Llama 3.2 model and generates an answer. |
| Automated Testing | tests/ — pytest | Tests individual components and the integrated retrieval and answer-generation workflow. |

The components are organized into separate modules to improve code organization, maintainability, and testing.

## 3. Execution Instructions

The application was developed using Python, uv, and Streamlit. The following steps describe how to run it in the development environment.

**Step 1: Open the project directory**

Open the project in VS Code and navigate to the project root directory.

**Step 2: Install project dependencies**

Run the following command in the terminal:

```
uv sync
```

**Step 3: Start Ollama**

Ensure that Ollama is installed and running, and that the required models are available:

```
ollama list
```

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

The application uses the following models:

- llama3.2 for answer generation and nomic-embed-text for embedding generation.

If either model is not available, download it using the following commands.

```
ollama pull llama3.2
```

```
ollama pull nomic-embed-text
```

**Step 4: Run the Streamlit application**

From the project root directory, execute:

```
uv run streamlit run src/app.py
```

Streamlit opens the application in a web browser. The user can upload an academic PDF, enter a question, and review the generated answer and retrieved document passages.

## 4. Evidence of Successful Execution

The integrated application was executed using representative academic research PDFs to verify the main functionality.

### 4.1 Successful PDF Processing and Question Answering

A research paper was successfully uploaded through the Streamlit interface. The application extracted its text, divided it into chunks, generated embeddings, and stored the document data in ChromaDB.

A natural-language question was submitted:

"What are the two dominant areas of human responsibility under the compressed SDLC?"

The application retrieved relevant passages from the research paper, including information about orchestration and verification. Llama 3.2 generated an answer based on the retrieved content.

### 4.2 Unsupported and Incomplete Input Handling

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

Two invalid-input scenarios were tested:

- An empty question: The application displayed the message, "Please enter a question."
- A scanned PDF without selectable text: The application displayed a warning that no selectable text was found, and that the PDF might contain scanned pages.

These tests demonstrated that the application provides feedback for these unsupported or incomplete inputs.

## 4.3 Execution Evidence

The following screenshots should be included:

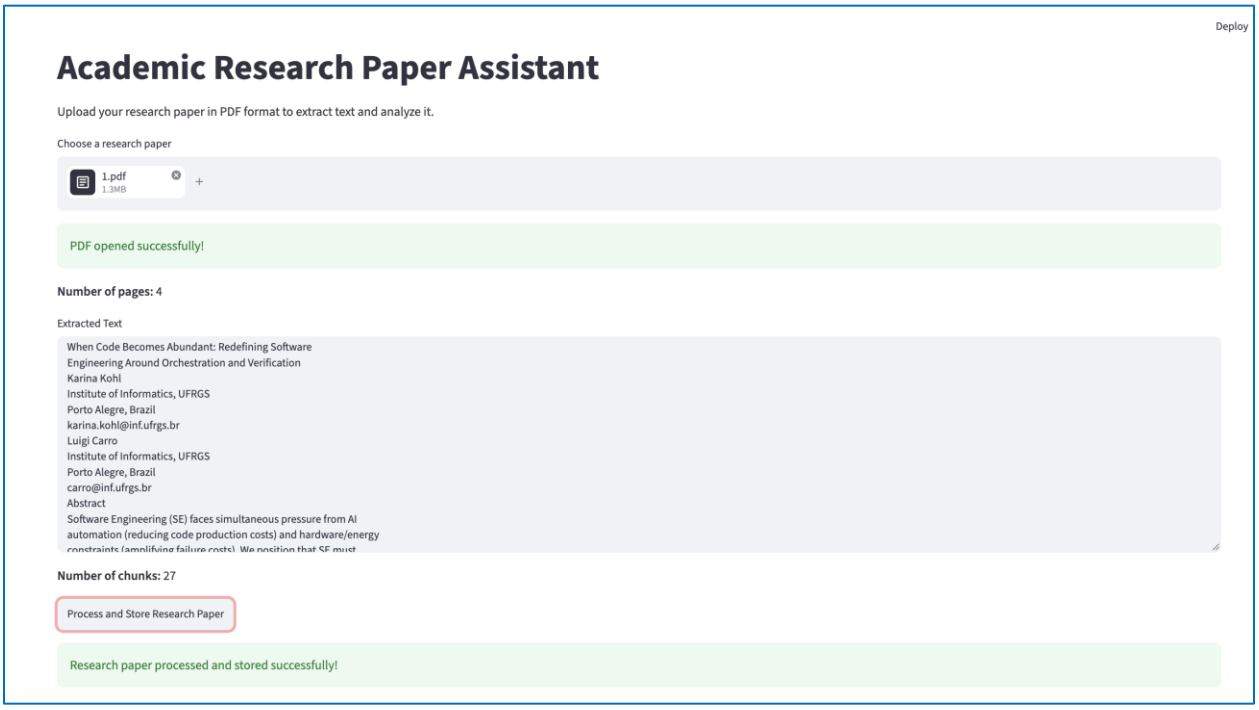



**Figure 1. Successful PDF Upload, Text Extraction, and Processing**

# Step 2: Software Development Project
# Week 3 - Component Integration
# and Initial Working System

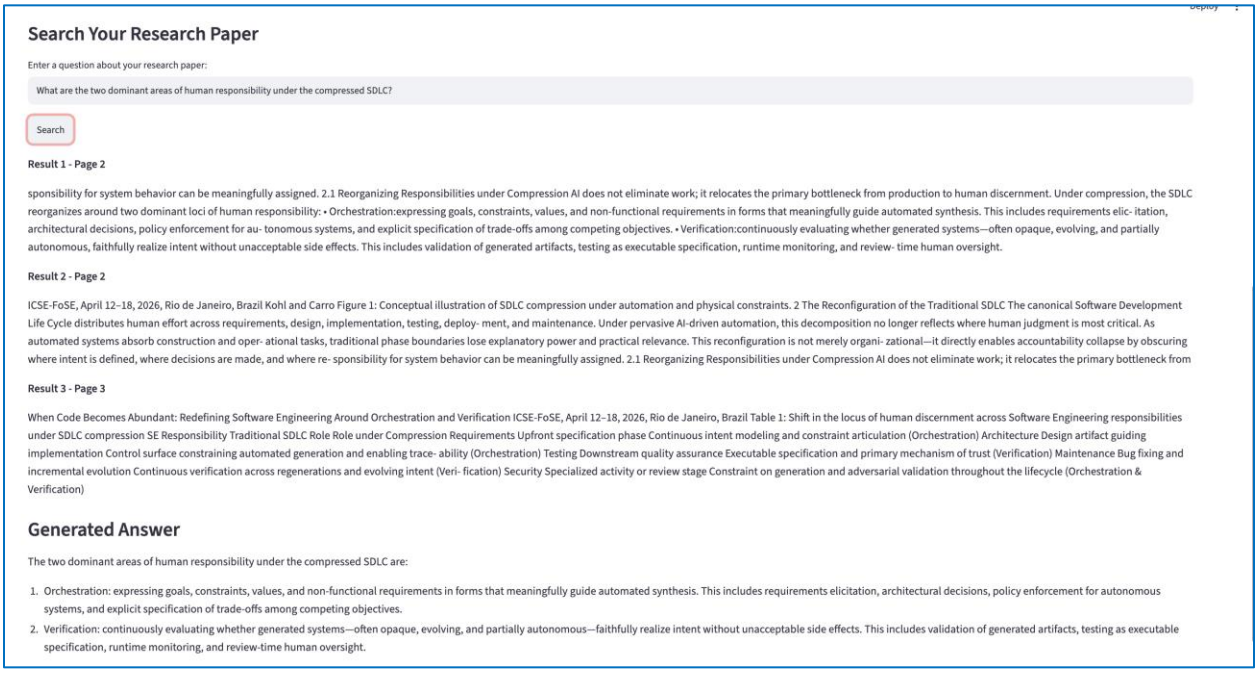



**Figure 2. Retrieved Research Paper Passages with Page Numbers and Generated Answer**

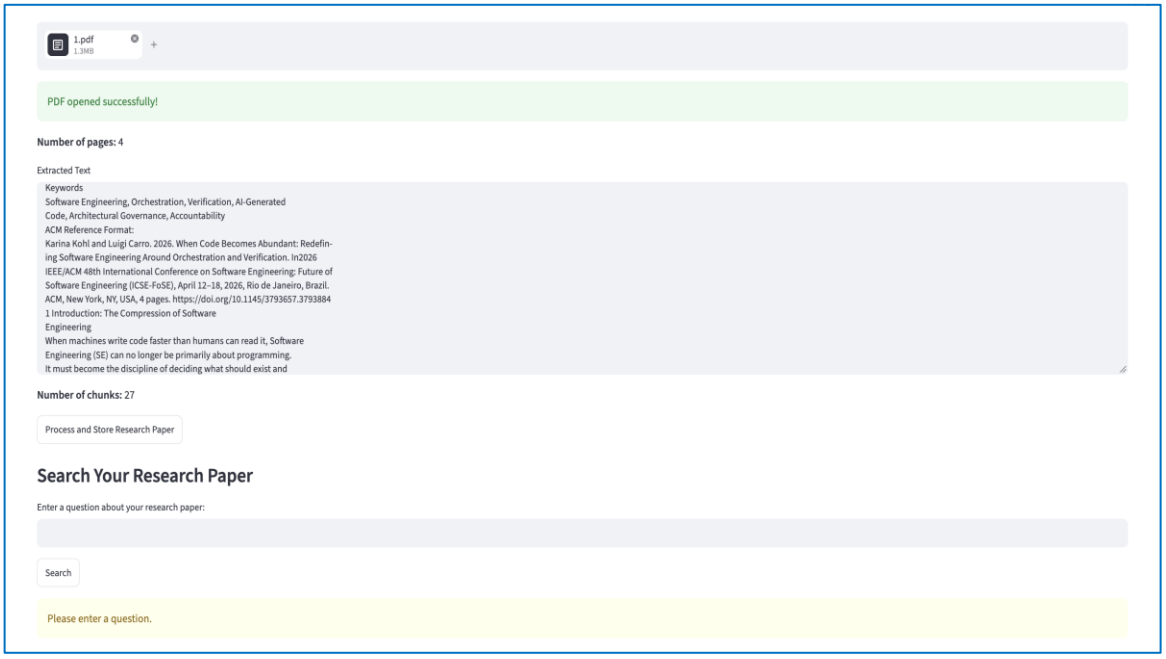



**Figure 3: Empty-question validation message**

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

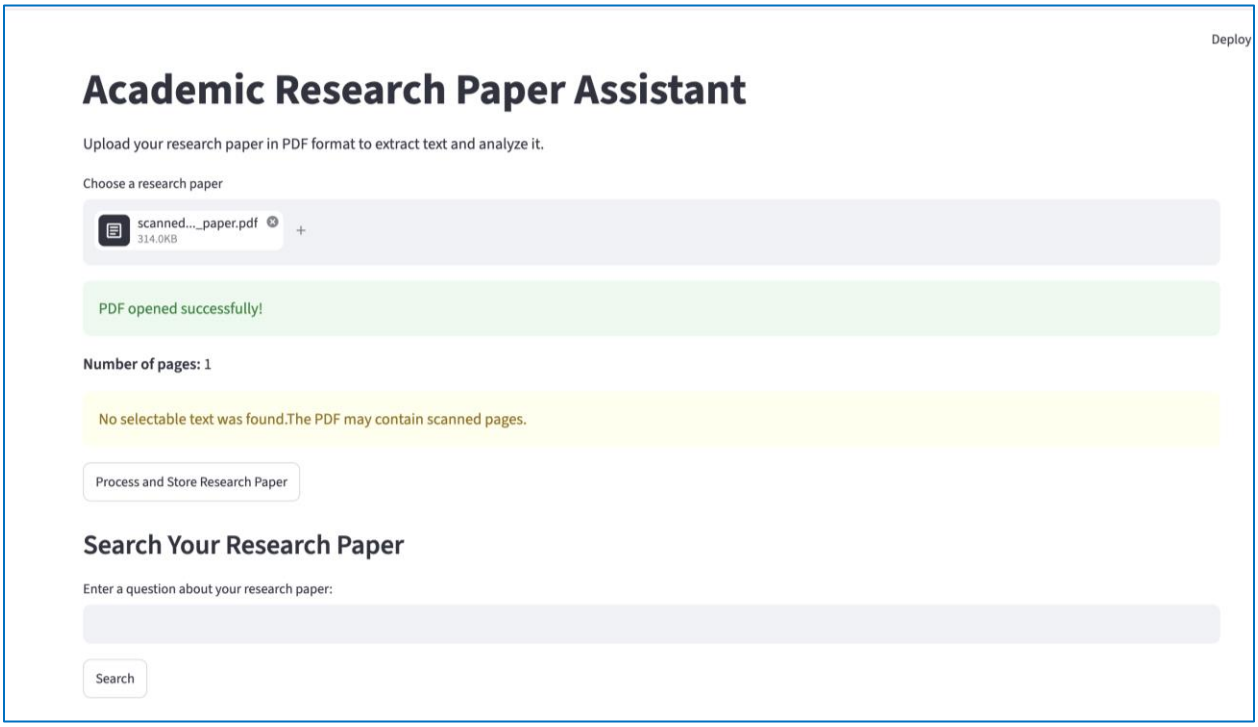



**Figure 4: Warning displayed when processing a scanned PDF without selectable text**

## 5. Integration Tests Summary and Results

Automated testing was performed using pytest to verify the individual components and their integration.

### 5.1 Automated Test Results

The test suite included 11 tests covering the following areas:

| Test File | Tests | Purpose |
|---|---|---|
| test_pdf_loader.py | 2 | Verify PDF text extraction and page information. |
| test_text_splitter.py | 2 | Verify text chunking and handling of empty pages. |
| test_embedding.py | 1 | Verify embedding model initialization. |
| test_vector_store.py | 4 | Verify vector store creation, storage, retrieval, and empty input handling. |
| test_answer_generator.py | 1 | Verify answer generation using the Llama 3.2 model. |
| test_rag_integration.py | 1 | Verify that retrieval and answer generation work together. |

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

### 5.2 Integration Test

An integration test was added to verify the interaction between ChromaDB retrieval and answer generation. A sample document passage was stored, a question was submitted, and the relevant passage was retrieved with its page's metadata. The answer-generation component was then tested using a mocked LLM response.

The test verified that the retrieved content was passed to the answer-generation component and that the expected answer was returned.

### 5.3 Test Execution Results

The complete test suite was executed using:

uv run pytest -v

**Result:** All 11 automated tests passed successfully.

This indicates that the tested components and their integration are functioning as expected under the tested conditions

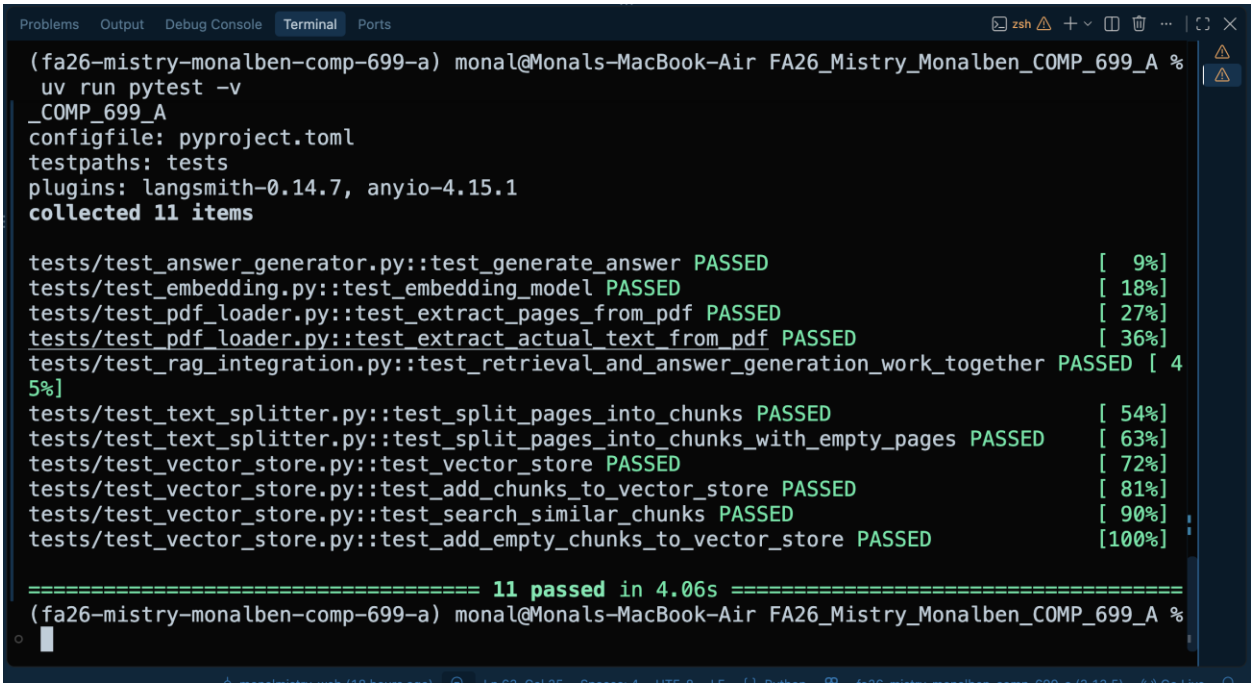



**Figure 5. Automated Test Execution Results Using Pytest**

## 6. Functional Requirements Status

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

The implementation was evaluated against the functional requirements defined in the original project architecture document.

| Requirement | Description | Week 3 Status |
|---|---|---|
| FR-01 | The system shall allow users to upload academic PDF documents through the web interface | Implemented |
| FR-02 | The system shall validate uploaded files and reject invalid or unsupported PDFs. | Partially Implemented |
| FR-03 | The system shall extract text from uploaded PDFs. | Implemented |
| FR-04 | Divide extracted text into smaller chunks | Implemented |
| FR-05 | Generate embeddings for document chunks | Implemented |
| FR-06 | Store embeddings and associated metadata | Implemented |
| FR-07 | Accept natural-language questions | Implemented |
| FR-08 | Retrieve relevant document chunks | Implemented |
| FR-09 | Provide retrieved content to the LLM as context | Implemented |
| FR-10 | Generate answers based on retrieved context | Implemented |
| FR-11 | Display generated answers | Implemented |
| FR-12 | Display source passages and page information | Partially Implemented |
| FR-13 | Display appropriate error messages | Partially Implemented |
| FR-14 | Provide a web interface for uploads and questions | Implemented |

### Progress Summary

By the end of Week 3, the main PDF processing, embedding, retrieval, and answer-generation requirements have been implemented and integrated.

FR-02, FR-12, and FR-13 require further work and testing. In particular, answer-level page citations and broader error handling have not yet been fully evaluated.

## 7. Design Changes and Unresolved Issues

### 7.1 Design Changes

No major changes were made to the original system architecture. The implementation follows the planned two-step RAG approach.

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

During the PoC, the embedding model was finalized as nomic-embed-text through Ollama. The remaining technology choices and architectural components stayed consistent with the original design.

### 7.2 Unresolved Issues and Future Improvements

The following areas require further work to meet the original project requirements and improve system quality:

- **PDF Processing:** Scanned PDFs without selectable text are not currently supported. OCR is outside the initial project scope.
- **Retrieval Relevance:** Improve retrieval quality so questions return relevant passages.
- **Answer Traceability:** Ensure generated answers to identify supporting document passages and page numbers when available.
- **Error Handling:** Expand testing and handling for invalid or corrupted PDFs, unavailable models, and unexpected processing failures.
- **Performance Evaluation:** Record document-processing and answer-generation times under normal operating conditions.
- **Usability and Reliability:** Review the interface and test the complete application under different input conditions.

The existing two-step RAG workflow will remain the core system. The project will first prioritize completing and evaluating the functionality and nonfunctional requirements defined in the original project plan.

## 8. Next Development Tasks

The following tasks are planned for the next development phase:

1. **Improve Answer Traceability:** Ensure generated answers include supporting page references and can be verified against the retrieved passages.
2. **Strengthen Error Handling:** Test and improve handling of invalid PDFs, corrupted files, unavailable Ollama models, and unexpected application errors.

# Step 2: Software Development Project
# Week 3 - Component Integration and Initial Working System

3. **Evaluate Retrieval and Answer Quality:** Test the application with different academic research papers and representative questions to assess retrieval of relevance and answer accuracy.

4. **Perform Performance and Usability Evaluation:** Record document-processing and answer-generation times, review the user interface, and evaluate application reliability.

5. **Final Integration and Documentation:** Review all functional and nonfunctional requirements, address remaining issues, update project documentation, and prepare for the final system demonstration.

If sufficient development time remains after completing the planned requirements, an agent-based retrieval workflow will be developed using LangGraph. The agent will evaluate retrieved information and determine whether another retrieval attempt is needed. The enhanced workflow will be tested and compared with the existing two-step RAG implementation.